CHAPTER 1 – Computational Complexity

Data structure – is a way of organizing and storing data so it can be assessed, changed, or searched properly. Its three benefits are efficiency, clarity, and scalability.

An algorithm – is a step-by-step set of instructions for solving a problem or performing tasks. (input -> algorithm -> output)

What makes a good algorithm?

- Input
- Output
- Definiteness
- Finiteness
- Effectiveness

Pseudocode – is a way of writing algorithms using simple, structured English-like statements, without worrying about the exact syntax of a programming language. (language-independent)

KEYWORD	MEANING
`START / END`	Where the algorithm begins and finishes
`INPUT / READ`	Take a value from the user
`PRINT / OUTPUT`	Show a result
`←`	Give a value to a variable: total ← 0
`IF … THEN … ELSE`	Make a decision
`FOR … TO … DO`	Repeat a known number of times
`WHILE … DO`	Repeat while a condition is true

AN EXAMPLE

```
Step 1: START
Step 2: total ← 0
Step 3: FOR i ← 1 TO n DO
Step 4:     total ← total + i
Step 5: END FOR
Step 6: PRINT total
Step 7: END
```

Indentation shows what belongs inside the loop — exactly as it does in Python.

Pseudocode Review — Is It Prime?

A number is prime if no number from 2 up to n − 1 divides it evenly.

PSEUDOCODE

```
Step 1: START
Step 2: INPUT n
Step 3: SET isPrime ← True
Step 4: FOR each i from 2 up to (but LESS THAN) n DO
            IF n MOD i = 0 THEN
                PRINT "The number n is not prime"
                SET isPrime ← False
                BREAK out of loop
            ENDIF
        ENDFOR
Step 5: IF isPrime = True THEN
            PRINT "The number n is prime"
        ENDIF
Step 6: END
```

PYTHON

```
n = int(input("Enter number: "))
isPrime = True

for i in range(2, n):
    if n % i == 0:
        print(f"The number {n} is not prime")
        isPrime = False
        break

if isPrime:
    print(f"The number {n} is prime")
```

BREAK stops the loop as soon as one divisor is found , there is no need to check the rest.

Flowchart — The Five Symbols

A flowchart draws the same steps as a picture.

	**Process**	A step that does something
	**Start / End**	Where the algorithm begins or stops
	**Input / Output**	Reading a value or printing a result
	**Decision**	A question with a Yes and a No branch
	**Flow direction**	The order the steps are carried out

Computational Complexity – a way of measuring how efficient an algorithm is in terms of two things:

1. Time complexity – how long it takes. (operations)

2. Space complexity – how much memory it uses. (cells)

Complexity is always measured relative to n, the size of the input.

Upper Bound – Big-O - worst case

Average – Big Theta – average case

Lower Bound – Big Omega – best case

Asymptotic Analysis – describes how complexity grows as n becomes larger.

CHAPTER 2 – Arrays, List & Strings

Arrays:

- Items side by side in memory;
- Fixed size – not dynamic;
- One type of values;

Python lists:

- No fixed size – dynamic;
- Different type of values;

Append at some index, removing by value – O(n) - needs shifting;

Append at the end, removing last item – O (1);

Strings:

- Sequence of characters;
- Immutable;

List Comprehension:

CODE	RESULT	MEANING
`[x for x in range(10) if x % 2 == 0]`	`[0, 2, 4, 6, 8]`	even numbers

CHAPTER 3 – Stack

Stack:

- Linear data structure;
- LIFO = Last In First Out;
- Examples: Undo (Ctrl + z), Browser Back Button, Recursive Functions, Pile of dishes.

Stack Operations:

- Push (x) - on top;
- Pop (x) - from top;
- Peek () or Top () - see the last item;
- IsEmpty( ) - check if it is empty;
- Search (x) - find x-item.

Underflow = empty

Overflow = full

CHAPTER 4 – Queue

Queue:

- Linear data structure;
- FIFO – First In First Out;
- Examples: Bank queues, Food orders, Supermarket checkout line.

Queue operations:

- Enqueue () - at rear;
- Dequeue () - from front;
- Peek () - see the first item;
- IsEmpty () - check if it is empty;
- Size () - count elements.

Four Types of Queues:

1. **Simple Queue.**
2. **Circular Queue.**
3. **Double-ended Queue (Deque).**
4. **Priority Queue.**

Python's lists are fast at one end only; a queue needs both ends.

Python list -> Pop (0) -> O(n) -> slow.

Deque (double-ended queue):

- Data Structure, Linked memory blocks;
- Made for queues: both ends are fast;
- Popleft () - >updates font reference -> no shifting -> O (1).

Queue – Round Robin:

- The CPU gives every program a short turn. An unfinished program goes back to the queue.

TURN	CPU RUNS	QUEUE AFTER THE TURN	WHAT HAPPENED
start	—	A · 5 | B · 2 | C · 3	*name · time still needed*
1	A for 2	B · 2 | C · 3 | A · 3	A still needs 3 → back of the line
2	B for 2	C · 3 | A · 3	**B finished**
3	C for 2	A · 3 | C · 1	C still needs 1 → back of the line
4	A for 2	C · 1 | A · 1	A still needs 1 → back of the line
5	C for 1	A · 1	**C finished**
6	A for 1		**A finished**

Time slice = 2. Run order: A B C A C A

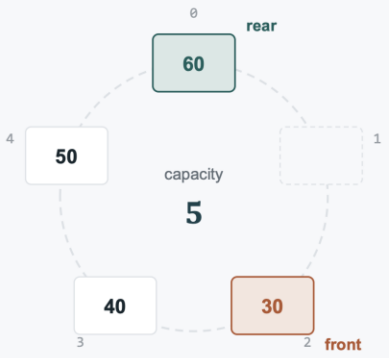
Dequeue a task, run it for one slice, and enqueue it again if it is not finished. Nobody waits forever.

Queue in a fixed-sized array (circular queue):

Circular Queue — The Idea

Join the end of the array to its start. After the last index comes index 0.

THE SAME ARRAY, DRAWN AS A RING



60 arrived after 50 — it wrapped around into the free slot at index 0.

The wrap-around formula

next = (index + 1) % capacity

% gives the remainder after division.

index	(index + 1) % 5	next
0	1 % 5	1
1	2 % 5	2
2	3 % 5	3
3	4 % 5	4
4	5 % 5	0

After index 4 the formula gives 0 — back to the start.

Circular Queue — Step by Step

capacity = 5. We remember front and size. A new item goes to index (front + size) % capacity.

STEP	OPERATION	ARRAY	0	1	2	3	4	front	size	WHY
1	`enqueue 10 … 50`		10	20	30	40	50	0	5	The array is full.
2	`dequeue() → 10`			20	30	40	50	1	4	front moves to index 1.
3	`dequeue() → 20`				30	40	50	2	3	front moves to index 2.
4	`enqueue(60)`		60		30	40	50	2	4	(2 + 3) % 5 = 0 → index 0
5	`enqueue(70)`		60	70	30	40	50	2	5	(2 + 4) % 5 = 1. Full again.
6	`dequeue() → 30`		60	70		40	50	3	4	front moves to index 3.

The clay box is the front. Queue order now, front to rear: 40, 50, 60, 70.

Can two LIFO stacks behave like one FIFO queue?

- Pouring one stack into another reverses the order.
  LIFO + LIFO = FIFO

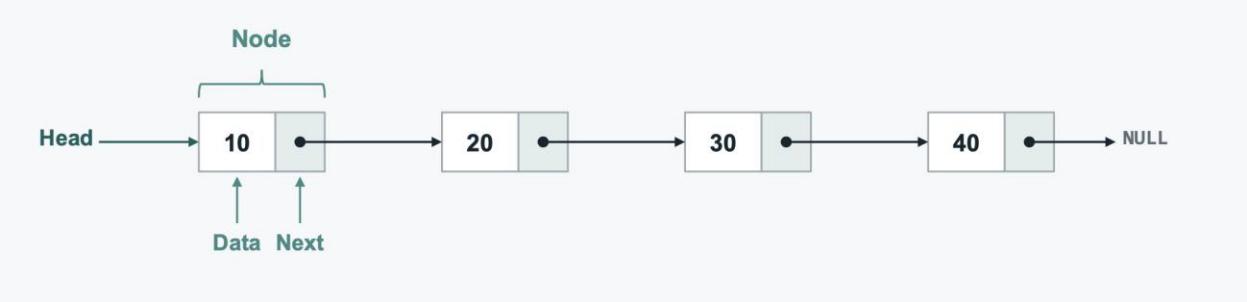
CHAPTER 5 – Linked List

Linked List - a sequence of nodes where each node stores

- Data;
- A link to the next node.

Linked List is different from an array with these three properties:

- Dynamic size (can grow or shrink);
- No contiguous memory (not side-by-side elements);
- Sequential access (no random index jumps).


Node
Head
10
20
30
40
NULL
Data
Next